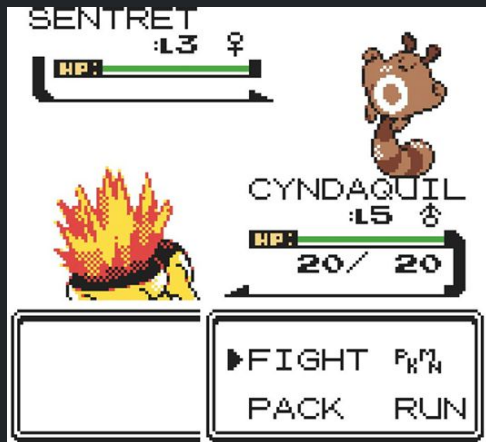


GMD2

Lecture 09: Pokemon

Pokemon



Today's Goal

VIA Mon!



Press Enter

Today's Topics

- State Stack
- Tweening
- GUIs
- Turn-Based Systems
- RPG Mechanics
- Service Locator

Source Code

- All Source code for today's game can be found here:
 - <https://github.com/Metamate/gmd2-pokemon>
- Walkthrough in README.md

Explore the Codebase

Take ~10 minutes to browse the code. Use the README as a guide if you need it.

- Clone the repository and build and play the game
 - Go through a few encounters in order to level up your mon!
- How is the codebase split between the core library and the pokemon-specific project?

State Stack

- Push/Pop states
 - Only the top state receives Update
 - Though this could be changed if we wanted it to
- All states draw, bottom to top
 - States underneath stay visible
 - Allows us to render multiple states at once
- This lets us layer screens on top of each other
 - Fade transitions — push a colour overlay on top, animate it, then pop
 - Dialogue boxes — message state sits on top of the battle scene
 - Battle phases — menu → turn → message, each pushes/pops independently
- Example battle stack: PlayState → BattleState → BattleMenuState
- Allows us to return back to states as they were when we left them
 - Resuming from a battle where we left the field (PlayState)

State Stack Trace

Files: StateStack.cs, BattleState.cs, BattleMenuState.cs, FadeState.cs

- A battle just triggered
 - List every state on the stack (bottom to top) once BattleMenuState is showing.
- Which state gets Update calls? Which states still Draw?
- FadeState pushes on top, animates, then pops
 - What would break if it swapped the current state instead of push/pop?
- Design a PauseState
 - where do you push/pop it?

Tweening System

- Smoothly changes a value over time (e.g. move a sprite from X=0 to X=100)
- Three ways to use it:
 - Tween — change a number from A to B over a set time
 - After — wait N seconds, then run a method
 - Every — repeat a method at a fixed interval (use .Limit to stop after N times)
- Chaining: .Add() to animate multiple values at once, .Finish() to run code when done
- Callbacks can also push/pop states or change game data
- Accessed via Service Locator: Locator.Tweens



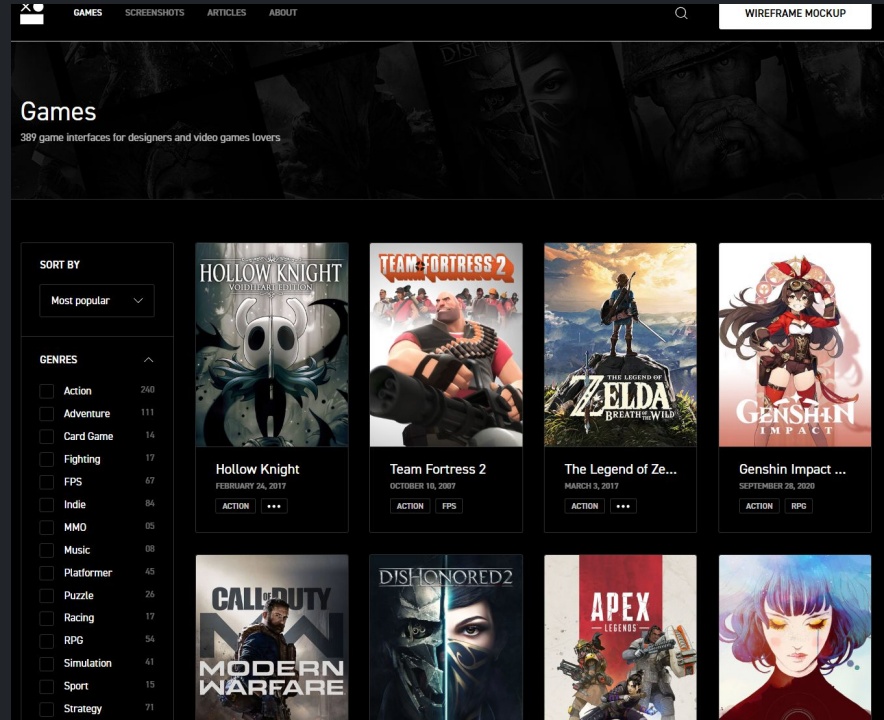
GUIs

Short for "graphical user interface"

Common widgets and elements include

- Panel
 - White rectangle with a black border, used as a background for all UI boxes
- ProgressBar
 - A bar that fills based on current/max (used for HP and EXP)
- Textbox
 - Wraps text inside a Panel, advances pages when player presses Confirm
- Selection / Menu
 - A list of options, each with a label and an action to run
 - Handles up/down navigation and cursor drawing — adding a new option is one line
- And others.. (labels, scrollbars, etc..)

GUIs



interfaceingame.com

Tile-Based Movement

- Two sets of coordinates per entity
 - MapX/MapY (int) — which tile the entity is on (used for game logic)
 - X/Y (float) — the pixel position on screen (smoothly tweened between tiles)
- EntityWalkState — handles one step between tiles
 - Snaps MapX/MapY to the new tile instantly, then tweens X/Y to catch up
- PlayerWalkState — if a direction key is still held, starts another step; otherwise goes Idle



Overworld & Random Encounters

- Level (Pokemon/World/Level.cs)
 - Two TileMap layers: base terrain + tall grass
 - Base terrain tile pattern is randomly generated
- Random encounter check
 - Runs each time the player ends a step in tall grass tiles
 - A random roll decides if a battle starts (probability set in GameSettings)
- Transition to battle:
 - Stop field music → start battle music
 - Fade to white → push BattleState → fade back in
 - All done with push/pop on the state stack — no special-case code

Overworld Encounters

- How does a battle start?
 - Find out where in the code the game transitions from PlayState to BattleState
 - What other states are involved in the StateStack at this moment?
 - What happens when a battle ends?



The Battle System

- BattleState
 - Draws the battle scene: two sprites, HP/EXP bars, bottom panel
 - Opening animation: both sprites slide in from opposite edges
- BattleMenuState
 - Shows the Fight / Run menu
- TakeTurnState
 - Runs one full battle round
 - Faster Pokemon (higher Speed) attacks first
 - Attack sequence: pause → lunge forward → hit sound → blink → HP bar drops
 - Each step is a tween that triggers the next one when it finishes — no if/else chain
- Flow: Menu → TakeTurn → (both alive?) → back to Menu, or → fade out

RPG Mechanics

- **PokemonSpecies**
 - Defines a species: name, base stats, growth rates, sprite paths
- **Mon**
 - One actual Pokemon in the game
 - Each level, every stat rolls a 6-sided die 3 times — higher growth rate = more likely to gain a point
- **Damage**
 - $(\text{Attack} * \text{move.BasePower} / 10) - \text{defender.Defense}$ (minimum 1)
- **Experience:**
 - Beating a Pokemon gives EXP based on its growth rates $\times$ level
 - `LevelUp()` returns how much each stat increased so the UI can show it
- **Party**
 - Holds your team of Mons; `Party.Current` is the one in battle

Monster Battles

- What is the flow of a single turn in battle?
 - From choosing an action to showing the result
- How are attacks and moves represented in the code?
 - Where is damage calculated?
- How does the game decide when a battle ends?
 - What happens right after victory or defeat?
- How are core RPG stats handled for a monster?
 - HP, IV, level, attack, experience, etc..
- What causes a monster to gain experience and level up
 - How are stat increases applied?
- How is a party of monsters modeled?
 - How does the battle system use that party data?

Definition Files

- Game data lives in JSON files, game logic lives in C#
- `Pokemon_definitions.json`
 - Species name, base stats, growth rates, sprite paths
 - Loaded into C# objects using `System.Text.Json`
- `Entity_animations.json`
 - Animation name, which spritesheet frames to play, timing
- The C# classes that receive this data use record types
 - Small classes that just hold data
- Adding a new Pokemon means editing a JSON file, not rewriting C# code

Data-Driven Definitions

- Add a new species to `pokemon_definitions.json`
 - Make it a glass cannon (high attack, low defense). Run the game and fight it
- Add a "type" field (Fire/Water/Grass) to the JSON and `PokemonSpecies.cs`
 - Verify the data loads correctly
 - Make Mon attacks take defender type into account (super effective vs not very effective)



Service Locator

- Many classes need shared services (audio, tweens, assets)
 - How do they find them?
- Solution: `Locator.cs` — a static class that holds references to shared services
 - Any class can call `Locator.Audio.PlayHit()` or `Locator.Tweens.Tween(...)`
 - Services are registered once at startup: `Locator.Provide(audioInstance)`
- Null object: before the real service is registered, a "do-nothing" version stands in
 - `NullAudio` implements `IAudio` with empty methods — no null checks needed
- `MonoGame` has `Game.Services`
 - Just a generic dictionary
 - E.g. `Services.AddService<IAudio>(audio)` and `Services.GetService<IAudio>()`
 - ... But not needed, as we roll our own

Some Missing Features

- Detailed Level-Up Screen
- Monster Catching
- Field Menu For Browsing Pokemon
- Item Inventory
- Different Abilities
- Other Trainers to Fight
- Monster Evolution
- Towns, Routes, and Other Levels
- Breeding
- Day/Night Cycle
- Types (Fire/Water/Grass)

Project Status

Short status meeting with each group

